Grammar:

Grammar is nothing but the set of rules use to define the lang. or a simply a grammar is define by four terms (V_N, Σ, P, S) or (V_N, T, P, S) where,

(1) V_N is the finite, non-empty set of Variables Or non-terminals.

(2) Σ is finite, non-empty set of constants or terminals.

(3) $V_N \cap \Sigma = \phi$

(4) $S \in V_N$ and known as start symbol

(5) P is finite, non-empty set of production rules

Following observations are consider regarding the production rules:

(1) **Reverse substition is not permitted:** If $S \to AB$ is a production then we can replace S by AB. But we cannot replace AB by S.

(2) **No inversion op. is permitted:** If $S \to AB$ is a production, it is not necessary that $AB \to S$ is a production.

Notation Used:

We will use following notations:-

1) All Capital letter A, B, C -- are use as variables
2) All Small letter a, b, c -- are use as terminals
3) x, y, z, w denote string of terminals.
4) α, β, γ denote the elements of $(V_N \cup \Sigma)^*$ i.e, denote strings of terminals or variable or both including null string.

(5) Any symbol to the power 0 is equivalent to ε (null) i.e, $x^0 = \varepsilon$ where $x \in (V_N \cup \Sigma)$

(6) If A is any set, then A^* denotes the set of all strings over A. A^+ denotes $A^* - \{\varepsilon\}$, where ε is the empty string.

#Derivation & the lang. generated by a Grammar:

If $\alpha \to \beta$ is a production in Grammar G, then $\alpha \underset{G}{\Rightarrow} \beta$ is called one step derivation or production. If $\alpha \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \alpha_3 - - \Rightarrow \alpha_n \Rightarrow \beta$ then $\alpha \overset{n}{\underset{G}{\Rightarrow}} \beta$ is called n step derivation. The notation $\underset{G}{\Rightarrow}$ represents relation for Grammar G.

Ex: Consider a Grammar G define as $G = (\{S\}, \{a, b\}, \{S \to asa, S \to \varepsilon\}, S)$.

Soln We have, $V_N = \{S\}$

$\Sigma = \{a, b\}$

$P = \{S \to asa, S \to \varepsilon\}$ and starting symbol S.

$S \underset{G}{\Rightarrow} asa$ (one step derivation)

$S \underset{G}{\Rightarrow} aasaa$ (two step derivation)

$S \underset{G}{\Rightarrow} aaasaaa$ (three step derivation)

$S \overset{n}{\underset{G}{\Rightarrow}} a^n S a^n$ (n-step derivation)

$S \underset{G}{\Rightarrow} a^n a^n$ (using production $S \to \varepsilon$)

$S \Rightarrow a^{2n}$

$L = \{a^{2n} \mid n > 0\}$

$L = \{\varepsilon, aa, aaaa - -\}$.

Ques A Grammar $G = (\{S\}, \{a, b\}, \{S \to aS, S \to \varepsilon\}, S)$
Describe the lang. generated by G.

Soln we have, $V_N = \{S\}$
$\Sigma = \{a, b\}$
$P = \{S \to aS, S \to \varepsilon\}$ and starting symbol S.

$S \underset{G}{\Rightarrow} aS$ (one step derivation)
$S \underset{G}{\Rightarrow} aaS$ (two " ")
$S \underset{G}{\Rightarrow} aaS$ (three " ")
.
.
.
$\underset{G}{\overset{n}{\Rightarrow}} a^n S$ (n step derivation)
$\underset{G}{\Rightarrow} a^n$ (using production $S \to \varepsilon$)

$L(G) = \{a^n \mid n \geq 0\}$

Ques If $G = (\{S\}, \{0, 1\}, \{S \to 0S1, S \to \varepsilon\}, S)$.

Soln we have, $V_N = \{S\}$
$\Sigma = \{0, 1\}$
$P = \{S \to 0S1, S \to \varepsilon\}$ and starting symbol S.

$S \underset{G}{\Rightarrow} 0S1$ (one step derivation)
$S \underset{G}{\Rightarrow} 00S11$ (two " ")
$S \underset{G}{\Rightarrow} 000S111$ (three " ")
.
.
.
$S \underset{G}{\overset{n}{\Rightarrow}} 0^n S 1^n$ (n step derivation)
$\underset{G}{\Rightarrow} 0^n 1^n$ (using production $S \to \varepsilon$)

$L = \{0^n 1^n \mid n \geq 0\}$

Ques If $G = (\{S\}, \{0,1\}, \{S \to SS\}, S\}$ Find $L(G) = ?$

Soln $L(G) = \phi$

since the only production $S \to SS$ in G has no terminal ~~has no terminal~~ on right hand side.

Ques If $G = (\{S, C\}, \{a, b\}, P, S\}$ where P consist of $S \to aca, C \to aca | b$ Find $L(G)$

Soln We have, $V_N = \{S, C\}$

$P = \{S \to aca, C \to aca|b\}$.

$\Sigma = \{a, b\}$.

S is starting symbol.

$S \underset{G}{\Rightarrow} aca$

$S \underset{G}{\Rightarrow} aacaa$

$S \underset{G}{\Rightarrow} aaacaaa.$

$S \underset{G}{\Rightarrow} a^n c a^n$

$S \underset{G}{\Rightarrow} a^n b a^n$

$L(G) = \{a^n b a^n | n > 0\}$.

Ques Let $G = (\{S, A_1, A_2\}, \{a, b\}, P, S)$, where P consist of $S \to aA_1A_1a; A_1 \to baA_1A_2b, A_2 \to A_1ab, aA_1 \to baa, bA_2b \to abab$ Test whether $W = baabbabaaaabba$ is in $L(G)$.

Sol^n $S \to aA_1A_1a$

$\Rightarrow baaA_1a$ [using $aA_1 \to baa$]

$\Rightarrow baabaA_1A_2ba$ [using $A_1 \to baA_1A_2b$]

$\Rightarrow baabaaA_2ba$ [using $aA_1 \to baa$]

$\Rightarrow baabaaA_1abba$ [using $A_2 \to A_1ab$]

$\Rightarrow baabbabaaaabba$ [using $aA_1 \to baa$].

#Derivations:

The Production rules are used derive certain string. If $G = (V_N, \Sigma, P, S)$ be some context free Grammar then Generation of some lang. using specific rules are called derivations

There are two types of derivation:-

- **Left Most Derivation (LMD):**

If $G = (V_N, \Sigma, P, S)$ is a CFG and $w \in L(G)$, then a derivation $S \overset{*}{\underset{lm}{\Rightarrow}} w$ is called a left most derivation if and only if all steps involved in derivation have left most variable replacement.

- **Right Most Derivation (RMD):**

If $G = (V_N, \Sigma, P, S)$ is a CFG and $w \in L(G)$, then a derivation $S \overset{*}{\underset{RM}{\Rightarrow}} w$ is called a Right most derivation if and only if all steps involved in derivation have rightmost variable replacement.

Que Consider the production rule of a Grammar 'G': $S \to S+S \mid S*S \mid a \mid b$. Find the left most and rightmost derivation for the string $w = a*a+b$

Sol^n Using left most derivation for $w = a*a+b$

$S \overset{}{\underset{lm}{\Rightarrow}} S*S$

$S \overset{}{\underset{lm}{\Rightarrow}} a*S$ (using $S \to a$)

$S \overset{}{\underset{lm}{\Rightarrow}} a*S+S$ (using $S \to S+S$)

$S \overset{}{\underset{lm}{\Rightarrow}} a*a+S$ (using $S \to a$)

$S \overset{}{\underset{lm}{\Rightarrow}} a*a+b$ (using $S \to b$)

Using Rightmost derivation for w = a*a+b

$S \underset{Rm}{\Rightarrow} S+S$ (using ~~using~~ .

$S \underset{Rm}{\Rightarrow} S+b$ (using $S \to b$)

$S \underset{Rm}{\Rightarrow} S*S+b$ (using $S \to S*S$)

$S \underset{Rm}{\Rightarrow} S*a+b$ (using $S \to a$)

$S \underset{Rm}{\Rightarrow} a*a+b$ (using $S \to a$)

#Derivation Tree:

Derivation Tree is a graphical representation for the derivation of the given production rules for a given CFG. The derivation tree is also called parse tree. Following are the properties of any derivation tree:-

(1) The root node is always a node indicating start symbol.

(2) The derivation is read from left to right.

(3) The leaf nodes are always terminal nodes.

(4) The interior nodes are always the non-terminal nodes.

(5) The collection of leaves from left to right yields the string w.

Ques Consider production rules of a Context free grammar: $S \to S+S \mid S*S \mid a \mid b$
Construct derivation tree for string $w = a*a+b$

Soln

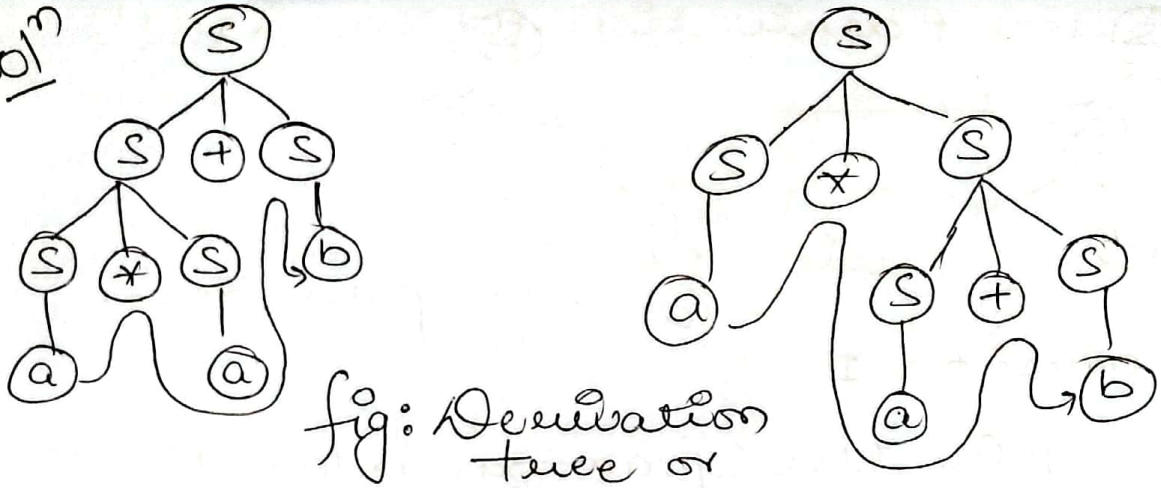


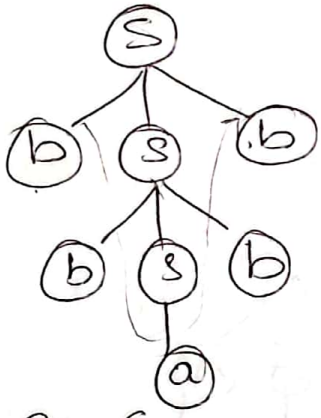
fig: Derivation tree or Parse tree.

Ques $S \rightarrow bSb \mid a \mid b$.

S → Starting symbol

Construction derivation tree for string $w = bbabb$

Soln



Ambiguity in Grammar:

A Grammar G is said to be ambiguous if there exists some string $w \in L(G)$ for which there are two or more distinct D.T or there are two or more leftmost or rightmost derivation. Ambiguous Grammar are undesirable bcz multiple derivation trees provide multiple info. about a certain string. Ambiguity is a negative property of a Grammar.

Ques What is Ambiguity? Show that $S \rightarrow as \mid sa \mid a$ is an ambiguous grammar for string $w = aaa$

Soln

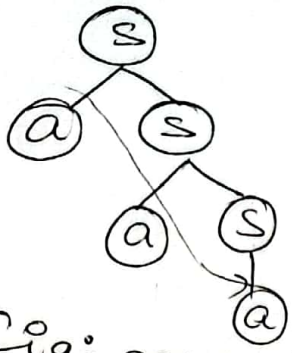


fig: parse tree 1

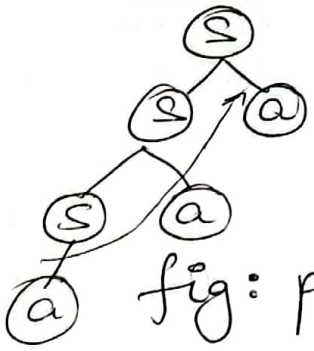


fig: parse tree 2.

Qu: Explain why the Grammar below that generate strings with an equal no. zeroes and one's is ambiguous

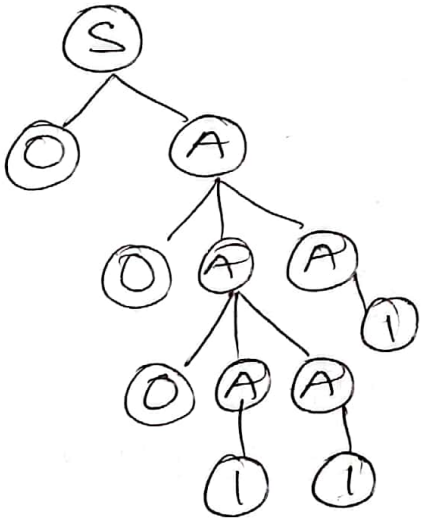
$S \to 0A/1B$

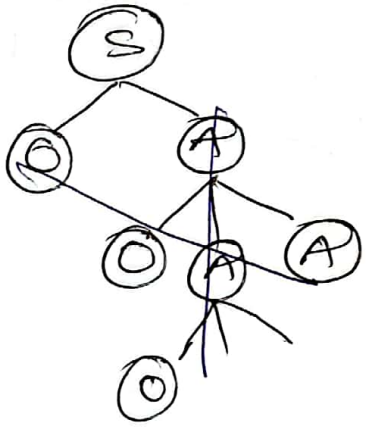
$A \to 0AA/1S/1$

$B \to 1BB/0S/0$.

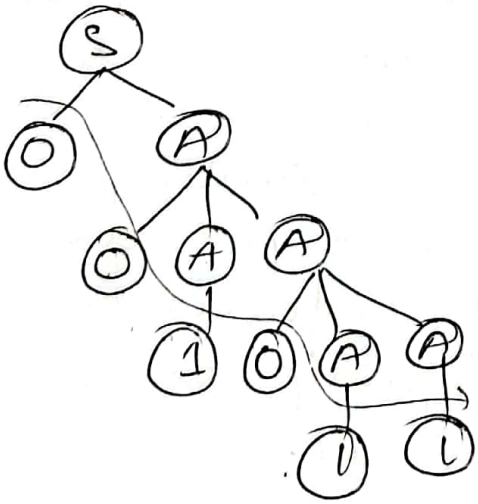
$w = 000111$

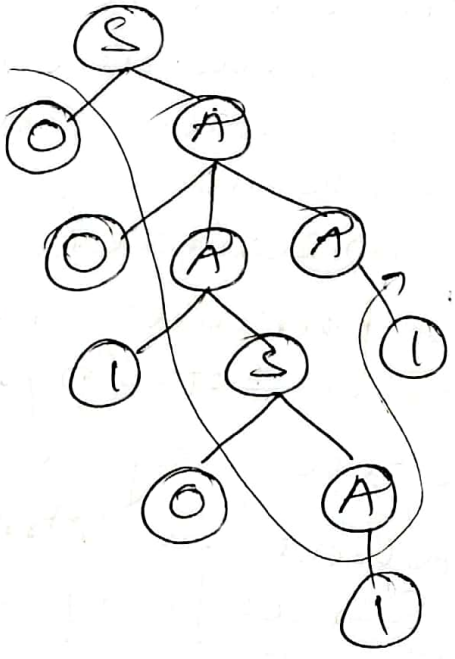
Soln





$w = 001011$





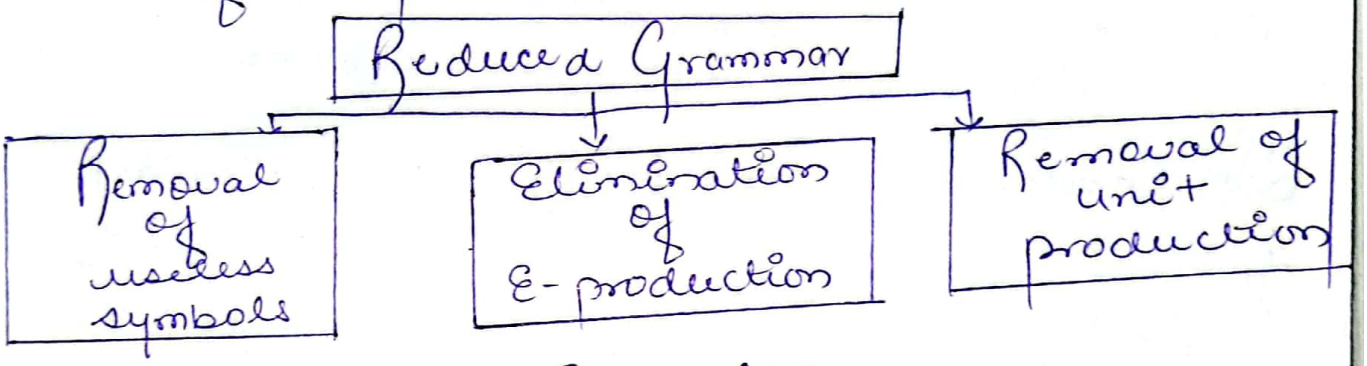
Simplification of CFG:

Simplification of Grammar means reduction of Grammar by removing useless symbols. The properties of reduced grammar are given below:

(1) Each Variable and each terminal of 'G' appears in the derivation of some string in 'L'.

(2) There should not be any production as $X \to Y$ where X and Y are non-terminals. ($X \to Y$ unit production)

(3) If ε is not in the lang. 'L' then there is no need of the production $X \to \varepsilon$



Removal of Useless Symbols:

Any symbol is useful when it appears on the right hand side in the production rule and generates some terminal string. If no such derivation exist then it supposed to be an useless symbol.

Ques → $G \Rightarrow (V_N, \Sigma, P, S)$ where $V_N = \{S, T, X\}$

$\Sigma = \{0, 1\}$

$S \to 0T \mid 1T \mid 0 \mid 1 \mid X$ (Rule 1)

$T \to 00$ (Rule 2)

Soln

$S \to 0T$	$S \to 1T$	$S \to 0$	$S \to 1$	$S \to X$
$S \to 00$	$S \to 100$	(useful)	(useful)	(useless)
(useful)	(useful)			

Remove $S \to X$

Reduced grammar is

$P = \{S \to 0T \mid 1T \mid 0 \mid 1$
$T \to 00\}$

$V_N = \{S, T\}$

$\Sigma = \{0, 1\}$

S → starting symbol.

Ques Find CFG with no useless symbol equivalent to -

$S \to AB \mid CA$
$B \to BC \mid AB$
$A \to a$
$C \to aB \mid b$

Soln

$S \to AB$ ✗	$S \to CA$ ✓	$B \to BC$ ✗	$B \to AB$ ✗	$A \to a$
$\to aB$	$\to ba$		$B \to AB$ ✗	$C \to b$ ✓
$\to aBC$			~~useful~~ useless.	useful
useless	useful	useless		

$$P = \begin{bmatrix} S \to CA \\ C \to b \\ A \to a \end{bmatrix}$$

$V_N = \{S, A, C\}$
$\Sigma = \{a, b\}$
S → Starting symbol.

a)

Ques Remove Useless symbol from the following Grammar.

Soln

$$P = \begin{bmatrix} S \to aA \\ A \to aA \mid a \\ D \to Ea \mid ab \\ E \to d \end{bmatrix}$$

$S \to aA \mid bB$
$A \to aA \mid a$
$B \to bB$
$D \to ab \mid \epsilon a$
$\epsilon \to a\epsilon \mid d$

Elimination of ε production :

Eg: $S \to 0S \mid 1S \mid \varepsilon$

$S \to 0S$
$S \to 0\varepsilon$
$S \to 0$

&

$S \to 1S$
$S \to 1\varepsilon$
$S \to 1$

To remove ε, we get after add 0 & 1.

$S \to 0, S \to 1$ $\quad S \to 0S \mid 1S \mid 0 \mid 1$ Ans.

Removal of Unit production :

The unit productions are the productions in which one non-terminal gives another non-terminal.

Ques Consider the CFG is as below :-

$S \to 0A \mid B \mid C$
$A \to 0S \mid 00$
$B \to 1 \mid A$
$C \to 01$, then remove the unit production.

66

Sol^n $S \to OA \mid 01 \mid 1B$
$A \to 0S \mid 00$
$B \to 1 \mid 0S \mid 00$
$C \to 01$

$S \to C$ ×
$B \to A$ ×

#Normal forms:

There are two important normal forms –

① Chomsky's Normal Form (CNF)

② Greibach Normal Form (GNF)

●CNF:

A Context free grammar is said to be in CNF if all its production rules are of type –

- Non-terminal → non terminal. And ~~one~~ of non-terminal
- NT → Terminal

Example → $A \to a\alpha$, where $\alpha \in V_N^*$
$A \to a$

$S \to \varepsilon$ is in 'G' if $\varepsilon \in L(G)$ when ε. We assume that S does not appear on the right hand side of any production.

Ques Convert the given CFG to CNF

$S \to asa \mid bsb \mid a \mid b$

Sol^n $S \to asa$
$S \to ASA$
$A \to a$
$S \to AA_1$ ✓
$A_1 \to SA$

$S \to bsb$
$S \to BSB$
$B \to b$
$S \to BB_1$ ✓
$B_1 \to SB$

$S \to AA_1$
$A_1 \to SA$
$A \to a$
$S \to BB_1$
$B_1 \to SB$
$B \to a$
$S \to a$
$S \to b$

is in CNF.

GNF:

A context free Grammar is said GNF if all its production rules are of type.

Non-terminal → one terminal. Any no. of non-terminal

$A \to \alpha a$, where $\alpha \in V_N^*$

$A \to a$, $\alpha = \varepsilon$

***To convert given CFG into GNF, two important lemma's are used:**

***Lemma 1 :-**

Let $G = (V_N, \Sigma, P, S)$ be a given CFG and if there is production $A \to \beta\gamma$ and $\beta = \beta_1 | \beta_2 | \cdots | \beta_n$

Then we can convert A rule to GNF as –

$A \to \beta_1\gamma | \beta_2\gamma | \beta_3\gamma | \cdots | \beta_n\gamma$, where, $\gamma \in V_N^*$

Ques Convert given CFG into GNF

Sol^n $S \to \underset{\beta}{A}\underset{\gamma}{A}$

$A \to \underset{\beta_1}{a}A | \underset{\beta_2}{b}A | \underset{\beta_3}{a}A\underset{\beta_4}{S} | \underset{\beta_4}{b}$

$$\left[\begin{array}{l} S \to aAA | bAA | aASA | bA \\ A \to aA | bA | aAS | b \end{array}\right]$$

Ques convert the CFG into GNF.

$S \to \underset{\beta}{C}\underset{\gamma}{A}$

$A \to a$

$C \to \underset{\beta_1}{a}B | \underset{\beta_2}{b}$

Sol^n

$$\left[\begin{array}{l} S \to a\beta A | bA \\ A \to a \\ C \to a\beta | b \end{array}\right]$$

Ques Convert the given grammar into GNF.

$S \to AB | BC$

$A \to aB | bA | a$

$B \to bB | CC | b$

$C \to c$

Soln $S \to aBB \mid bBC \mid bAB \mid ccc \mid BC \mid aB$
$A \to aB \mid bA \mid a$
$B \to bB \mid cc \mid b$
$C \to c$

*Lemma 2:-

Let $G = (V_N, \Sigma, P, S)$ be a given CFG, if there is a production.

$A \to A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid --- \mid A\alpha_n \mid B_1 \mid B_2 \mid B_3 \mid -- \mid B_n$ *

such that β_i do not start with A then equivalent grammar in GNF can be-

$A \to \beta_1 \mid \beta_2 \mid \beta_3 \mid -- \mid \beta_n.$
$A \to \beta_1 Z \mid \beta_2 Z \mid \beta_3 Z \mid -- \mid \beta_n Z$
$Z \to \alpha_1 \mid \alpha_2 \mid \alpha_3 \mid --- \mid \alpha_n.$
$Z \to \alpha_1 Z \mid \alpha_2 Z \mid --- \mid \alpha_n Z$

Ques Consider $A \to A_1 \mid 0B \mid 2$

Soln $\alpha_1 = 1$
$\beta_1 = 0B$
$\beta_2 = 2$

$A \to 0B \mid 2$
$A \to 0BZ \mid 2Z$
$Z \to 1$
$Z \to 1Z$

Ques Convert the following grammar in GNF.

Regular Grammar :

A grammar is regular if it has rules of the form.

$A \to a$ or $A \to aB$ or $A \to \varepsilon$

Linear Grammar :

A grammar is said to be linear if any non-terminal generates a single non-terminal on right hand side.

$A \to B$

Left Linear Grammar :

A Gram. is said to be left linear if the production contain a single non-terminal on RHS of production which occur at the left most of the string.

$S \to Sa$
$S \to Pb$ } left linear grammar.
$P \to a$

Right Linear Grammar :

A Gram. is said to be right linear if the production contain a single non-terminal on RHS of production which occur at the right most of the string.

$S \to aS$
$S \to bP$ } Right linear grammar
$P \to a$

Conversion of Regular Grammar into finite Automata :

No. of states in the automata will be equal to no. of non-terminals + 1. Each state in automata represents each non-terminal in regular gram. Additional state will be final state.

Rules :

(1) For every production, $A \to aB$ make $\delta(A, a) = B$
(make a edge labelled 'a' from A to B)

(2) for every production, $A \to a$ make $\delta(A, a) =$ final state.

(3) for every production, $A \to A$ or ε, make $(\delta A, A) = A$ and A will be final state.

Ques Consider the following regular gram.

$S \to 0A|1B|0|1$

$A \to 0S|1B|1$

$B \to 0A|1S$

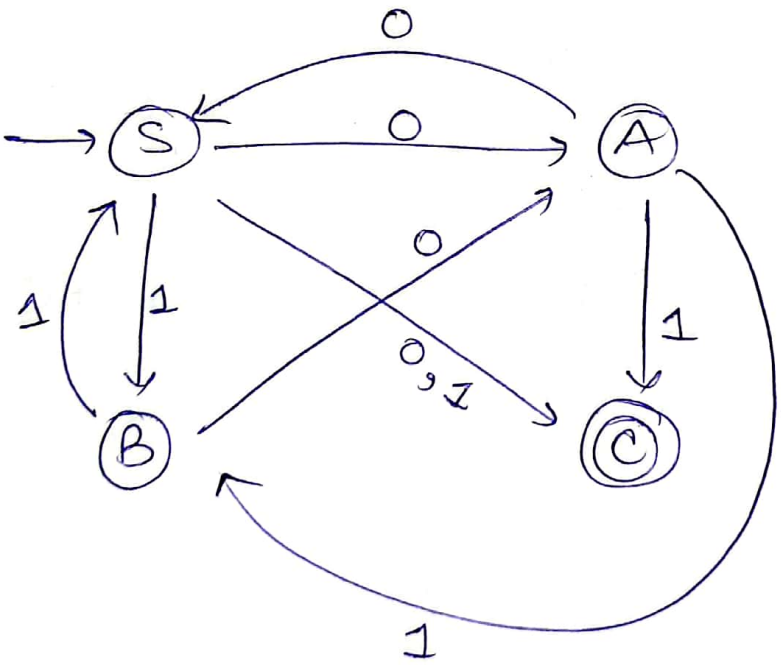
Design finite Automata

Soln

$S \to 0A|1B|0|1$

$A \to 0S|1B|1$

$B \to 0A|1S$



$S \to 0A$, $\delta(S, 0) = A$

$S \to 1B$, $\delta(S, 1) = B$

$S \to 0$, $\delta(S, 0) = C$

$S \to 1$, $\delta(S, 1) = C$

$A \to 0S$, $\delta(A, 0) = S$

$A \to 1B$, $\delta(A, 1) = B$

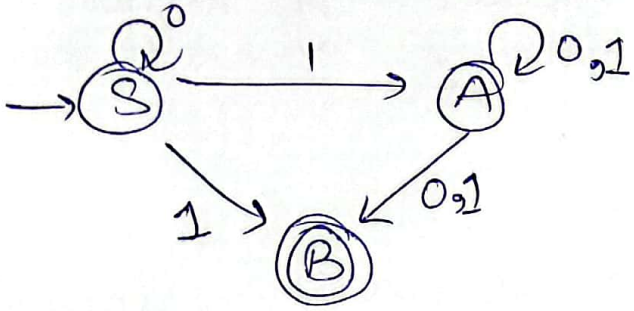
$A \to 1$, $\delta(A, 1) = C$

$B \to 0A$, $\delta(B, 0) = A$

$B \to 1S$, $\delta(B, 1) = S$

Ques Give the automata for the following regular grammar.

Soln $S \to 0S \mid 1A \mid 1$
$A \to 0A \mid 1A \mid 0 \mid 1$



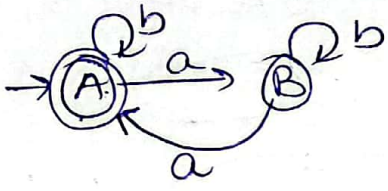
$S \to 0S$, $\delta(S, 0) = S$
$S \to 1A$, $\delta(S, 1) = A$
$S \to 1$, $\delta(S, 1) = B$
$A \to 0A$, $\delta(A, 0) = A$
$A \to 1A$, $\delta(A, 1) = A$
$A \to 0$, $\delta(A, 0) = B$
$A \to 1$, $\delta(A, 1) = B$

#Conversion of finite automata to RG or CFG:

Rules:

(1) FA: A —a→ B production: $A \to aB$
(2) FA: A —a→ B (B final) production: $A \to a$, $A \to aB$
(3) If initial state is final state, then add ε in a production

Ques Convert the following FA to RG



Soln $A \to \varepsilon$; $B \to bB$
$A \to bA$; $B \to aA$
$A \to aB$; $B \to a$
$A \to a$

RG: $A \to bA \mid aB \mid \varepsilon \mid b$
$B \to bB \mid aA \mid a$

Regular Grammar:
$G = (\{A, B\}, \{a, b, \varepsilon\}, P, A\}$
where, P: $A \to bA \mid aB \mid \varepsilon \mid b$
$B \to aA \mid bB \mid a$

#Chomsky's Hierarchy of Grammar:

Noam chomsky has classified grammar in 4 categories:-
(type 0, type 1, type 2 and type 3) based on the RHS forms of the productions. A Grammar may be subset or superset of another Grammar as per its types. A higher type grammar is superset of all lower types grammar.

● Type 0:

A Type 0 Gram. is any phrase structure grammar without any restrictions. All grammars are type 0 gram. and all lang. are type 0 lang. A production without any restriction is called a type zero production. Type 0 lang is accepted by turing m/c.

$$\phi A \psi \rightarrow \phi \alpha \psi$$

where, 'A' is variable, ϕ is called left context $\&$ ψ is the right context $\&$ α the replacement string.

Ex: (a) $\overset{\phi}{ab}A\overset{\psi}{bcd} \rightarrow \overset{\phi}{ab}AB\overset{\psi}{bcd}$

Soln $ab \rightarrow$ left context
$bcd \rightarrow$ Right context
$\alpha \Rightarrow AB$

(b) $\overset{\phi}{A}C\overset{\psi}{\varepsilon} \rightarrow \overset{\phi}{A}\overset{\alpha}{\varepsilon}\overset{\psi}{\varepsilon}$

Soln $A \rightarrow$ left context
$\varepsilon \rightarrow$ right context
$\alpha \Rightarrow \varepsilon$

(c) $C \rightarrow \varepsilon$

Soln $\varepsilon =$ left context
$\varepsilon =$ right Context
$\varepsilon = \alpha$

Type 1:

Restricted Type 0 gram. are known as Type 1 or context Sensitive gram (CSG). If all production of a Grammar are of type 1 production, then the Gram. is known as Type 1 Grammar. The language generated by a context Sensitive gram. is called a CSL. Type 1 lang is accepted by linear bounded automata (LBA). A production of the form $\phi A \psi \to \phi \alpha \psi$ is called Type 1 production. If $\alpha \neq \varepsilon$

(a) $aAbc \to abcDbc$ is a type 1 production where a, bc are the left context and right context respectively. A is replaced by $bcB \neq \varepsilon$.

(b) $AB\varepsilon \to AbBc\varepsilon$

$\alpha = bBC \neq \varepsilon$

$\phi = A$

$\psi = \varepsilon$

Eg: $S \to aA \mid \varepsilon$ is allow.

~~Eg~~: $S \to aAS \mid \varepsilon$ is not allowed.

Type 2:

A restricted Type 1 gram. are known as Type-2 gram. A Type 2 production is a production of the form $A \to \alpha$ where $A \in V_N$ & $\alpha \in (V_N \cup \Sigma)^*$ In other words, the LHS of productions has no left or right Context.

Ex: $S \to Ad$, $A \to a$, $B \to abc$, $A \to \varepsilon$ are type 2 production.

A Grammar is called a type 2 grammar if it contains only type 2 productions it is also called the content free Grammar. Push down automata can be used to represent these languages.

Type-3:

This is the most restricted type. A Gram. is c/d a type 3 or regular gram. If all it's production are type 3 productions. A production $S \to \varepsilon$ is allowed in type 3 gram. But in this case S does not appear on the RHS of any production. A production of the form $A \to a$ or $A \to aB$, where $A, B \in V_N$ (variables) and $a, b \in \Sigma$ (terminal) are c/d a type-3 production.